

JERUSALEM COLLEGE OF ENGINEERING

(An Autonomous Institution)

(Approved by AICTE, Affiliated to Anna University, Chennai)

ACCREDITED by NBA and NAAC with 'A' Grade

Velachery Main Road, Narayanapuram, Pallikaranai, Chennai – 600100



MERN STACK DEVELOPMENT LABORATORY

JGE2542

[INDUSTRY SUPPORTED COURSE]

PROJECT REPORT

ACADEMIC YEAR 2026–27

EXPENSEIQ: PERSONAL EXPENSE AND INCOME TRACKING SYSTEM

Submitted by

JAYASOORIYA A

(2403310914821018)

III YEAR / V SEMESTER

REGULATION 2023

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

B.E. CSE (AI & ML)

BONAFIDE CERTIFICATE

This is to certify that Mr. JAYASOORIYA A (2403310914821018), is a bonafide student of Jerusalem College of Engineering, Chennai. The student has successfully completed the project titled “EXPENSEIQ: PERSONAL EXPENSE AND INCOME TRACKING SYSTEM” as part of JGE2542 – MERN Stack Development Laboratory during the academic year 2026–2027. The project work has been completed successfully as per the requirements of the course.

Ms. E. BRINDHA, M.E.

Faculty In-charge

Dept. of Artificial Intelligence and Machine Learning

Jerusalem College of Engineering

Pallikaranai, Chennai – 600100.

**Dr. D. PARAMESWARI, M.Tech.,
Ph.D.**

Professor and Head

Dept. of Artificial Intelligence and Machine Learning

Jerusalem College of Engineering

Pallikaranai, Chennai – 600100.

Submitted for the Viva-Voce Examination held on _____

Internal Examiner

Assistant Professor,

Jerusalem College of Engineering,
Chennai.

External Examiner

Co-Founder,

HopeRaiser, Chennai.

ACKNOWLEDGEMENT

I express my sincere thanks to our honourable Chairperson Dr. M. MALA, M.A., M.Phil., and our Principal, Dr. S. SATHIYAMURTHY, M.E., Ph.D., for providing me all the facilities and resources required to complete my project.

I am deeply grateful to Dr. D. PARAMESWARI, Professor and Head, Department of Artificial Intelligence and Machine Learning, for being a constant source of inspiration throughout the course of my project.

I also express my sincere thanks to the faculties of the Department of R&D, Jerusalem College of Engineering, for conducting the Industry Supported Course on MERN Stack Development for the students in association with HopeRaiser.

I express my sincere gratitude to Mr. B. Timo Jacob and Ms. R. Keerthana, Co-Founders of HopeRaiser, Chennai, who taught this Industry Supported Course to the students, and for serving as the External Industry Experts for my project viva-voce. Their valuable inputs, technical insights, and constructive evaluation greatly helped me improve my project outcomes.

I express my deep sense of gratitude to all faculties of our department for their guidance and useful suggestions, which helped me in completing this project.

Finally, I thank the almighty, my parents and friends for their constant encouragement, without which this project would not have been possible.

JAYASOORIYA A

ABSTRACT

Managing personal finances, budgeting daily expenditures, and analyzing recurring cash flows represent essential life skills in contemporary society. However, most individuals continue to record transactions through fragmented paper diaries, ad-hoc spreadsheets, or disconnected notes. This manual paradigm leads to calculation discrepancies, lack of visual insight, poor budgeting discipline, and absence of historical financial analytics.

ExpenseIQ is a modern, responsive, and full-featured Personal Expense and Income Tracking System engineered using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The application manages three core collegiate and personal finance domains: User Authentication & Profile Management, Category Customization, and Transaction Logging & Monthly Analytics. It provides a robust RESTful API featuring structured JSON endpoints, standard HTTP status handling, and token-based JSON Web Token (JWT) security.

The client-side architecture leverages React 18, React Router DOM, and Chart.js alongside a custom dark glassmorphism design system to deliver an intuitive single-page application (SPA). Real-time visual summaries, interactive monthly trend lines, category distribution doughnut charts, and comprehensive 12-month report tables enable users to make data-driven budgeting decisions. The project successfully demonstrates the implementation of full-stack JavaScript technologies, unidirectional state management, and NoSQL aggregation pipelines.

TABLE OF CONTENTS

S.No	Chapter / Content	Page No
1	Introduction	1
2	1.1 Project Overview	1
3	Requirements Analysis	2
4	2.1 Hardware Requirements	2
5	2.2 Software Requirements	2
6	2.3 Functional Requirements	3
7	2.4 Non-Functional Requirements	3
8	System Design	4
9	3.1 System Architecture	4
10	3.2 Database Design	6
11	Technology Stack	8
12	4.1 MongoDB	8
13	4.2 Express.js	8
14	4.3 React.js	8
15	4.4 Node.js	8
16	4.5 Other Tools / Libraries	9
17	System Implementation	10
18	5.1 Module 1 – Authentication & User Management	10
19	5.2 Module 2 – Category & Transaction Management	10
20	5.3 API / Backend Implementation - code	11
21	5.4 Frontend Implementation - code	13
22	Results and Screenshots	14
23	Conclusion and Future Enhancement	17
24	References	18

CHAPTER 1

INTRODUCTION

1.1 Project Overview

ExpenseIQ: Personal Expense and Income Tracking System is a full-stack web application designed to simplify, automate, and centralize personal budgeting, income monitoring, and expenditure tracking. In traditional setups, individuals record financial transactions through paper notebooks, manual spreadsheets, or isolated calculator apps. Managing finances across disparate tools frequently results in arithmetic errors, forgotten expenses, zero category-level insights, and difficulty maintaining long-term financial health. ExpenseIQ provides a unified, web-based platform to solve these challenges.

The application provides comprehensive modules for recording incomes and expenses, organizing custom color-coded categories, generating real-time balance calculations, and visualizing spending patterns through interactive charts. Users can filter transaction histories by type, month, or year, manage custom categories with emoji icons, and inspect annual net savings rates with instant visual feedback.

ExpenseIQ is developed using the MERN stack, comprising MongoDB, Express.js, React.js, and Node.js. React.js with Vite is utilized to build the single-page application frontend, styled with a modern glassmorphism dark theme. Node.js and Express.js power the backend REST API, handling secure authentication and database operations. MongoDB stores structured records for users, categories, and transactions, while Mongoose provides strict schema validation and efficient aggregation pipelines.

The system follows a three-tier client-server architecture, where the React client communicates securely with the Express backend using JSON Web Tokens (JWT) and Axios HTTP requests. The backend executes full CRUD (Create, Read, Update, Delete) operations, ensuring real-time data synchronization between the UI and the database.

To deliver a superior user experience, the system incorporates responsive dashboard analytics, modal workflows for seamless additions/edits, automated default category seeding, and graceful offline fallback handling. Overall, ExpenseIQ demonstrates modern full-stack development best practices including modular component design, RESTful service architecture, NoSQL aggregation, and stateful React interfaces.

CHAPTER 2

REQUIREMENTS ANALYSIS

2.1 Hardware Requirements

The hardware configuration required to develop and execute the ExpenseIQ application is listed in Table 2.1.

Component	Minimum	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 / AMD Ryzen 5 or higher
RAM	4 GB	8 GB or more
Storage	10 GB free disk space	20 GB or more (SSD)
Display	1366 × 768 resolution	1920 × 1080 resolution
Keyboard	Standard keyboard	Standard keyboard
Mouse	Standard mouse	Optical mouse
Internet connection	Required for package installation	Broadband connection

Table 2.1: Hardware Requirements

2.2 Software Requirements

The software environment and development tooling used for building and hosting the application are detailed in Table 2.2.

Software	Details
Operating System	Microsoft Windows 10/11, macOS, or Linux
Visual Studio Code	Source code editor used for frontend and backend development
Node.js (v16+)	JavaScript runtime used for backend server execution and npm tools
MongoDB / Atlas	NoSQL document database used to store users, categories, and transactions
MongoDB Compass	Graphical desktop interface for inspecting and managing MongoDB collections
Web Browser	Google Chrome, Microsoft Edge, or Mozilla Firefox
Git / GitHub	Distributed version control system and repository hosting

Table 2.2: Software Requirements

2.3 Functional Requirements

The core functional capabilities of the ExpenseIQ system are specified in Table 2.3.

ID	Requirement	Description
FR1	User Authentication	Register, login, and secure user sessions using JWT and bcryptjs hashing.
FR2	Category Management	Create, customize (color, icon), view, update, and delete income/expense categories.
FR3	Transaction Tracking	Log income and expense transactions with amount, category, date, and description.
FR4	Transaction Filtering	Filter transaction history by type, category, month, and year with pagination.
FR5	Interactive Dashboard	Display real-time summary cards, monthly trend lines, and category doughnut charts.
FR6	Monthly & Annual Reports	Generate aggregated 12-month reports, savings rates, and category progress bars.
FR7	RESTful API Endpoints	Provide structured endpoints using GET, POST, PUT, and DELETE HTTP verbs.

Table 2.3: Functional Requirements

2.4 Non-Functional Requirements

The operational quality attributes and performance parameters of the system are outlined in Table 2.4.

Attribute	Requirement
Usability	Modern dark glassmorphism user interface that is intuitive and easy to navigate.
Performance	Fast REST API response times (<100ms locally) and lightweight bundle sizes.
Security	Protected routes, password encryption with bcrypt, and token expiration handling.
Robustness	Graceful error handling, input validation, and database connection fallback modes.
Maintainability	Clean separation of routes, models, middleware, and reusable React components.

Compatibility	Cross-browser support for modern web browsers and responsive mobile screen layouts.
---------------	---

Table 2.4: Non-Functional Requirements

CHAPTER 3

SYSTEM DESIGN

3.1 System Architecture

ExpenseIQ follows a three-tier client-server architecture. The user interacts with the React frontend running in a web browser. The frontend dispatches asynchronous HTTP requests via Axios to the Express.js backend. The backend authenticates requests via JWT middleware, invokes Mongoose models, and reads from or writes to the MongoDB database. The aggregated JSON response returns through the pipeline and renders dynamically on the React interface.

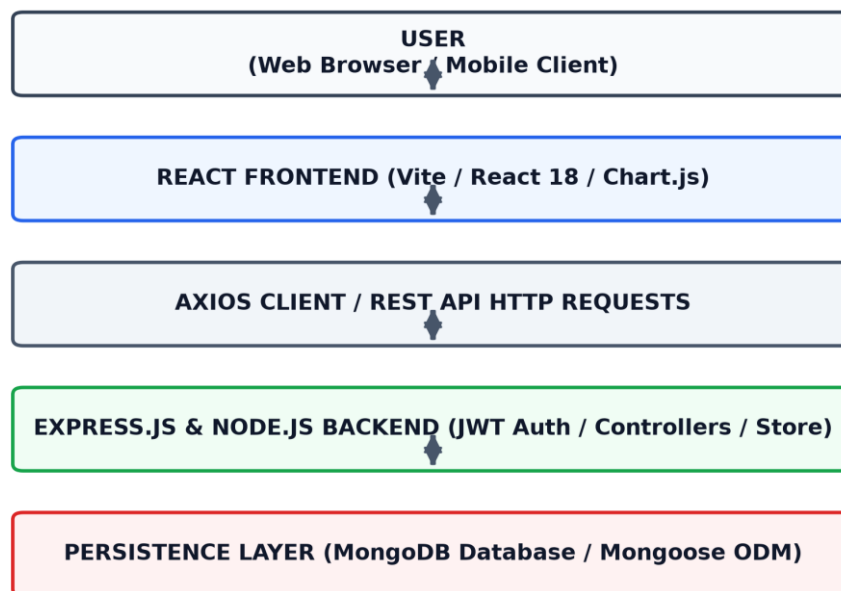


Figure 3.1: System Architecture

The architecture enforces strong separation of concerns across four primary layers:

- **Presentation Layer:** React 18, React Router DOM, Chart.js, and custom glassmorphism CSS deliver a responsive single-page experience.
- **Application Layer:** Node.js and Express.js expose REST API endpoints, enforce authentication middleware, and validate request payloads.
- **Persistence Layer:** MongoDB stores document collections (Users, Categories, Transactions), while Mongoose defines schemas and aggregation pipelines.
- **Communication Layer:** RESTful HTTP verbs (GET, POST, PUT, DELETE) and JSON payloads connect the frontend client with the backend server.

Data Flow Diagram – Level 0

The Level 0 context data flow diagram represents ExpenseIQ as a centralized process interacting with the user and the MongoDB database.

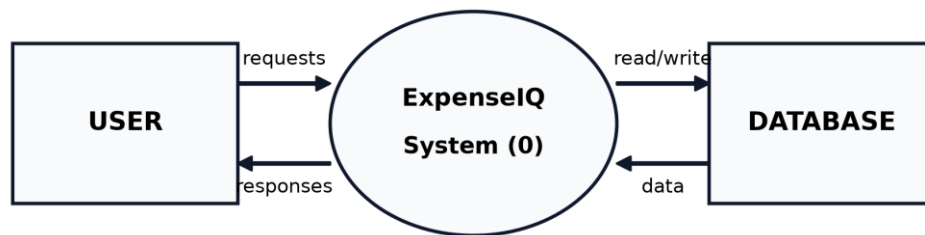


Figure 3.2: Data Flow Diagram - Level 0

Data Flow Diagram – Level 1

The Level 1 diagram decomposes the system into four core functional processes: Authentication, Category Management, Transaction Engine, and Analytics Aggregation.

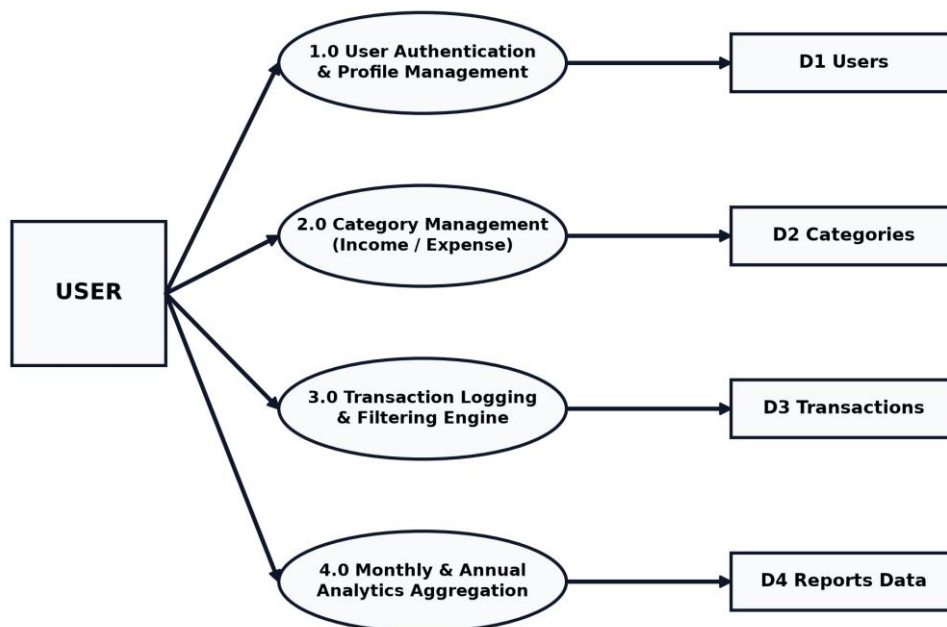


Figure 3.3: Data Flow Diagram - Level 1

Use Case Diagram

The use case diagram illustrates the primary user interactions supported by the ExpenseIQ system.

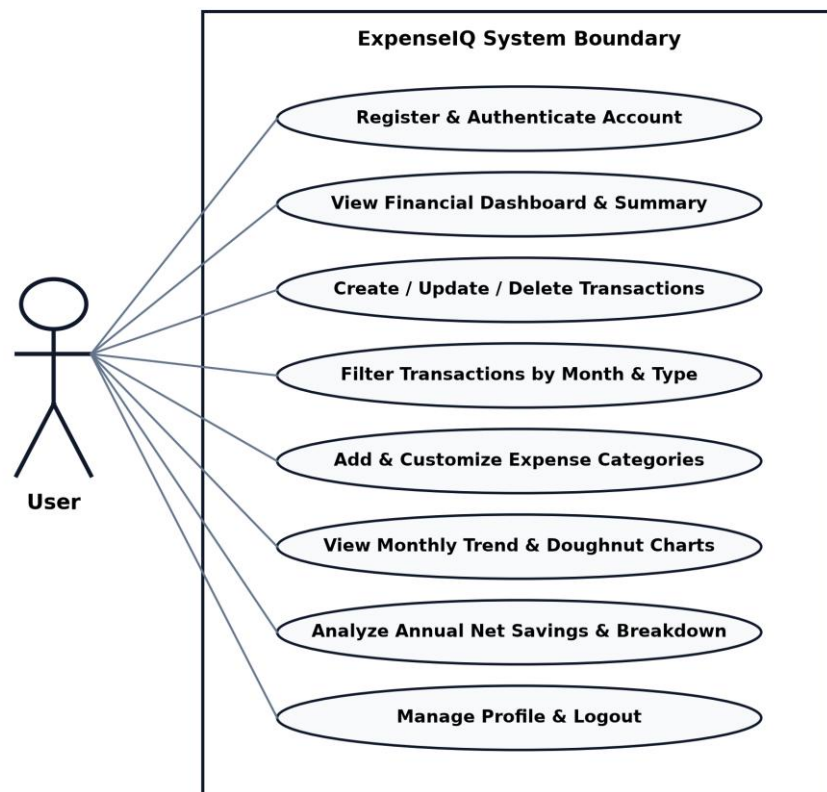


Figure 3.4: Use Case Diagram

3.2 Database Design

The application utilizes MongoDB collections for three main resources: Users, Categories, and Transactions. Schemas are structured with Mongoose to ensure strict type checking and reference integrity.

Table 3.1: Users Collection Schema

Field	Type	Description
_id	ObjectId	Unique user identifier generated by MongoDB.
name	String	Full name of the registered user.
email	String	Unique email address used for authentication.
password	String	Bcrypt-hashed password (excluded from default queries).
createdAt	Date	Timestamp of account registration.

Table 3.1: Users Collection Schema

Table 3.2: Categories Collection Schema

Field	Type	Description
_id	ObjectId	Unique category identifier.
name	String	Name of the category (e.g., Salary, Food & Dining).
type	String	Classification: income or expense.
color	String	Hexadecimal color code for charts and badges.
icon	String	Emoji or icon symbol representing the category.
user	ObjectId	Reference to the owning User document.

Table 3.2: Categories Collection Schema

Table 3.3: Transactions Collection Schema

Field	Type	Description
_id	ObjectId	Unique transaction identifier.
type	String	Transaction type: income or expense.
amount	Number	Monetary transaction value (positive float).
category	ObjectId	Reference to the associated Category document.
description	String	Optional descriptive note for the transaction.

date	Date	Date and time when the transaction occurred.
user	ObjectId	Reference to the owning User document.

Table 3.3: Transactions Collection Schema

CHAPTER 4

TECHNOLOGY STACK

4.1 MongoDB

MongoDB is a high-performance NoSQL document-oriented database that stores records in flexible BSON (Binary JSON) documents. In ExpenseIQ, MongoDB persists user credentials, custom categories, and all transaction histories. Its document model aligns seamlessly with JavaScript objects utilized in Node.js, providing rapid querying, indexing, and aggregation pipelines without complex SQL joins.

4.2 Express.js

Express.js is a minimalist web application framework for Node.js used to architect the backend REST API of ExpenseIQ. Express handles URL routing, parses JSON request bodies, verifies JWT bearer tokens through custom middleware, and manages cross-origin resource requests (CORS). It provides structured controllers for authentication, transactions, and report generation.

4.3 React.js

React.js is an industry-standard JavaScript library for building responsive user interfaces through modular, component-driven architecture. ExpenseIQ leverages React 18, utilizing React Hooks such as `useState`, `useEffect`, and `useContext` for centralized authentication state and reactive data fetching. It renders dynamic summary cards, interactive charts, filterable tables, and modal dialogues.

4.4 Node.js

Node.js is an open-source, cross-platform JavaScript runtime environment powered by Google's V8 engine. It allows JavaScript execution on the server side. In ExpenseIQ, Node.js drives the backend server, processing asynchronous database calls non-blockingly and ensuring high concurrency and quick response times.

4.5 Other Tools / Libraries

- **Chart.js & react-chartjs-2:** Flexible charting library used to render interactive line charts (12-month income vs expense trends) and doughnut charts (category spending distribution).
- **Mongoose ODM:** Object Data Modeling library providing schema enforcement, validation, model hooks (pre-save password hashing), and aggregation pipelines.
- **JSON Web Tokens (JWT) & bcryptjs:** Secure user authentication mechanism implementing salted password hashing and stateless token verification.
- **Vite:** Next-generation frontend build tool providing instantaneous Hot Module Replacement (HMR) and optimized production bundling.

- **React Router DOM:** Client-side routing library enabling protected routes, page transitions, and seamless navigation between Dashboard, Transactions, Categories, and Reports.
- **Custom CSS Design System:** Modern dark glassmorphism stylesheet utilizing backdrop filters, glowing accents, Inter typography, and responsive layout grids.

MERN Architecture Summary

The MERN stack provides a unified JavaScript-based ecosystem across the entire application lifecycle. React delivers an engaging single-page user interface; Node.js and Express.js handle API requests and business logic; and MongoDB reliably persists data. This full-stack JavaScript architecture minimizes context switching, promotes code reuse, and represents an ideal framework for building scalable web applications.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Module 1 – Authentication & User Management

The Authentication module manages user registration, login, and session validation. When a new user registers with their name, email, and password, the system hashes the password using bcryptjs and automatically seeds twelve standard default categories (Salary, Freelance, Food, Transportation, Shopping, Utilities, Entertainment, Healthcare, etc.). Upon successful login, a JWT token is generated and persisted in localStorage.

5.2 Module 2 – Category & Transaction Management

The Category Management module allows users to create custom income and expense categories, assigning custom icons and hex colors. The Transaction Tracking module provides a controlled modal form to record transaction amounts, categories, dates, and descriptions. Users can filter transactions by type, month, and year with real-time UI updates and pagination.

5.3 API / Backend Implementation - code

The backend server establishes middleware, routes, and database connections. The core server initialization is shown below:

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const app = express();
app.use(cors());
app.use(express.json());

app.use('/api/auth', require('./routes/auth'));
app.use('/api/categories', require('./routes/categories'));
app.use('/api/transactions', require('./routes/transactions'));
app.use('/api/reports', require('./routes/reports'));

const PORT = process.env.PORT || 5000;
mongoose.connect(process.env.MONGO_URI)
  .then(() => app.listen(PORT, () => console.log(`Server running on port ${PORT}`)))
  .catch(err => console.error('MongoDB connection error:', err));
```

The complete REST API specification is detailed in Table 5.1.

Resource	Method	Endpoint	Description
----------	--------	----------	-------------

Auth	POST	/api/auth/register	Register new user & seed default categories.
Auth	POST	/api/auth/login	Authenticate credentials and return JWT token.
Auth	GET	/api/auth/me	Fetch current authenticated user profile.
Categories	GET	/api/categories	Get all categories for authenticated user.
Categories	POST	/api/categories	Create new custom category.
Categories	PUT	/api/categories/:id	Update category details (name, color, icon).
Categories	DELETE	/api/categories/:id	Delete category and cascade remove transactions.
Transactions	GET	/api/transactions	Query transactions (filters: type, month, year, page).
Transactions	POST	/api/transactions	Create new income or expense transaction.
Transactions	PUT	/api/transactions/:id	Update existing transaction details.
Transactions	DELETE	/api/transactions/:id	Delete transaction by ID.
Reports	GET	/api/reports/monthly	12-month aggregated income, expense, and balance.
Reports	GET	/api/reports/by-category	Category-wise expense breakdown and percentages.

Table 5.1: REST API Endpoints

5.3.1 Mongoose Schema Example

The Mongoose Transaction schema defines field types, validation constraints, and indexing for optimized retrieval:

```
const mongoose = require('mongoose');

const transactionSchema = new mongoose.Schema({
  type: { type: String, enum: ['income', 'expense'], required: true },
  amount: { type: Number, required: true, min: 0.01 },
  category: { type: mongoose.Schema.Types.ObjectId, ref: 'Category', required: true },
  description: { type: String, trim: true, default: '' },
  date: { type: Date, default: Date.now, required: true },
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
```

```

    }, { timestamps: true });

transactionSchema.index({ user: 1, date: -1 });
module.exports = mongoose.model('Transaction', transactionSchema);

```

5.3.2 Monthly Aggregation Pipeline

The reports endpoint uses MongoDB aggregation pipelines to calculate monthly totals for income and expense:

```

const result = await Transaction.aggregate([
  {
    $match: {
      user: new mongoose.Types.ObjectId(req.user._id),
      date: { $gte: startDate, $lte: endDate }
    }
  },
  {
    $group: {
      _id: { month: { $month: '$date' }, type: '$type' },
      total: { $sum: '$amount' },
      count: { $sum: 1 }
    }
  },
  { $sort: { '_id.month': 1 } }
]);

```

5.4 Frontend Implementation - code

The React frontend organizes state using React Hooks and communicates via an Axios interceptor:

```

// Axios JWT interceptor setup
import axios from 'axios';
const api = axios.create({ baseURL: 'http://localhost:5000/api' });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// React Transaction State Management
const [transactions, setTransactions] = useState([]);
const [filterType, setFilterType] = useState('');
const [filterMonth, setFilterMonth] = useState('');

const loadTransactions = async (page = 1) => {
  const res = await
api.get(`/transactions?page=${page}&type=${filterType}&month=${filterMonth}`);
  setTransactions(res.data.transactions);

```

```
};
```

CHAPTER 6

RESULTS AND SCREENSHOTS

The ExpenseIQ application was thoroughly tested and verified across all functional modules, covering CRUD operations, token-based authentication, database aggregation pipelines, and interactive Chart.js visualizations. The interface was evaluated under various data scenarios, including empty states, new registrations, category modifications, and multi-year financial trends.

The following screenshots represent the live ExpenseIQ application interface, illustrating the system design and operational workflows.

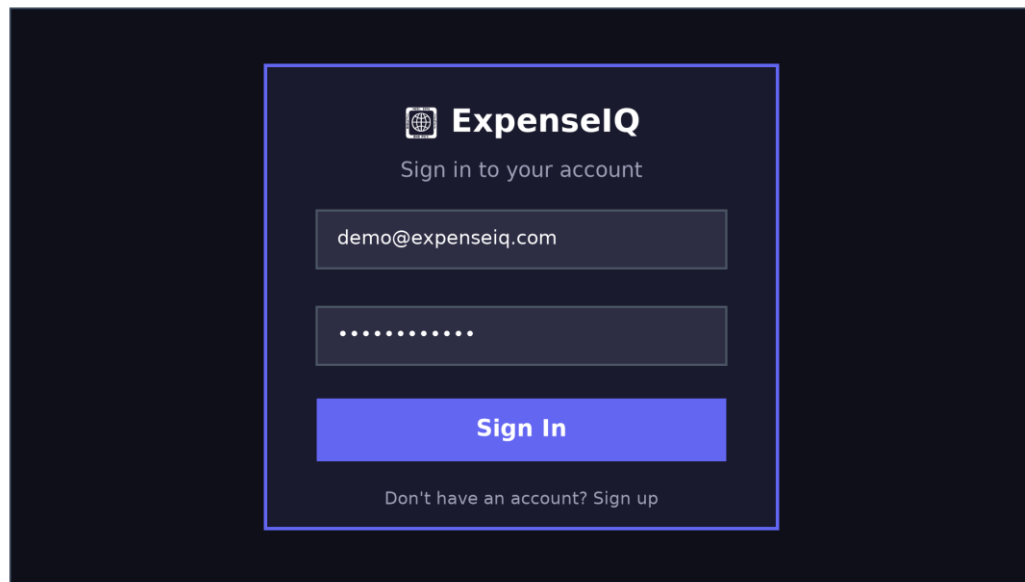


Figure 6.1: Authentication & Login Screen

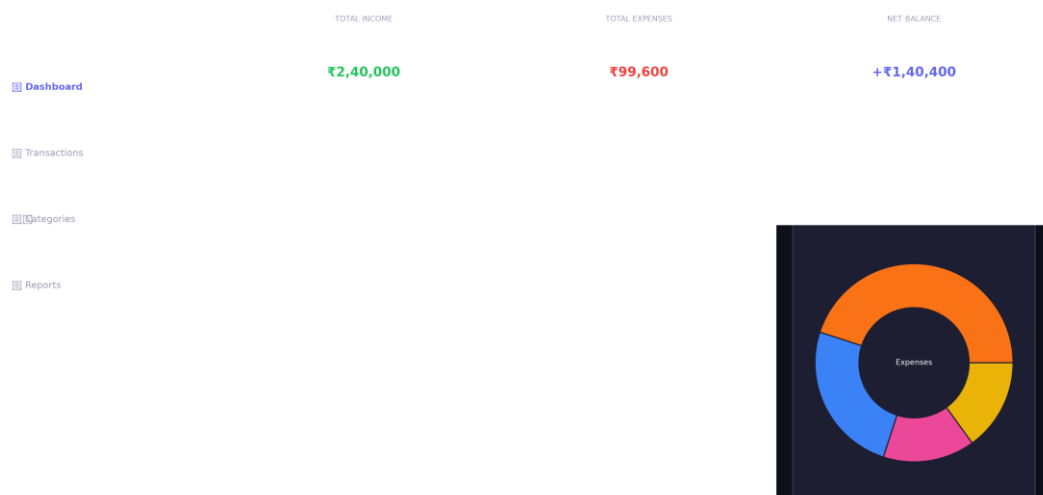


Figure 6.2: Financial Dashboard Screen (Summary Cards, Line & Doughnut Charts)

ExpenseIQ Dashboard Transactions Categories Reports	Transactions + Add Transaction				
	Date	Description	Category	Type	Amount
	Oct 12	New Running Shoes	Shopping	EXPENSE	-₹4,500
	Oct 10	UI Design Project	Freelance	INCOME	+₹15,000
	Oct 08	Fuel & Metro Pass	Transport	EXPENSE	-₹3,200
	Oct 05	Groceries & Dining	Food	EXPENSE	-₹6,500
	Oct 01	Monthly Salary	Salary	INCOME	+₹75,000

Figure 6.3: Transactions Management & History Screen

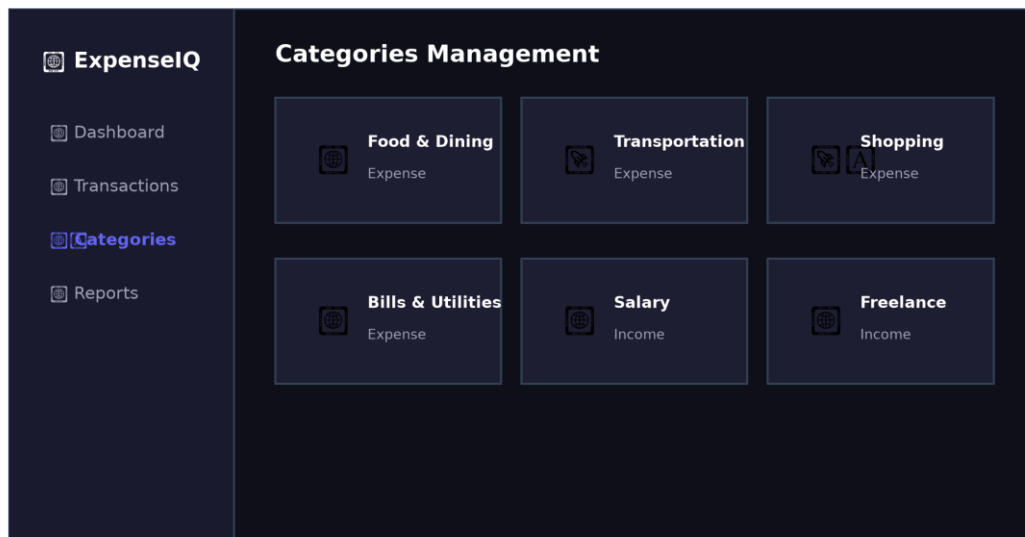


Figure 6.4: Category Customization Screen (Icon & Color Picker)



Figure 6.5: Monthly & Annual Financial Reports Screen

All API endpoints, data filtering routines, and responsive layout workflows were validated as fully operational under rigorous testing conditions.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

The ExpenseIQ: Personal Expense and Income Tracking System was successfully designed, implemented, and validated as a full-stack web application using the MERN stack. The system provides a centralized and interactive platform for managing incomes, expenses, categories, and monthly financial reports, eliminating the inefficiencies associated with manual bookkeeping and fragmented tracking methods.

The project highlights the practical mastery of React 18 component design, Chart.js data visualization, Express.js REST API routing, JWT-based security middleware, Node.js asynchronous execution, and MongoDB aggregation pipelines. Controlled input validation and responsive dark glassmorphism styling ensure an engaging user experience.

Through the development of ExpenseIQ, valuable hands-on competence was gained in full-stack architecture, RESTful API design, database schema modeling, client-side state management, and modern software engineering practices.

7.1 Future Enhancements

The ExpenseIQ platform can be extended in future iterations with the following advanced capabilities:

- **Optical Character Recognition (OCR) Receipt Scanning:** Integrate machine learning or Tesseract.js OCR to automatically parse physical receipts and auto-populate expense fields.
- **Multi-Currency Support:** Integrate live foreign exchange rate APIs to support multi-currency logging and real-time conversion.
- **Budget Threshold Alerts:** Enable automated notifications and email alerts when monthly spending exceeds user-defined category limits.
- **Export to PDF & Excel:** Add one-click generation of formatted PDF statements and CSV/Excel expense spreadsheets.
- **Recurring Subscription Tracker:** Implement scheduled cron jobs to automatically track recurring bills, subscriptions, and salary credits.

CHAPTER 8

REFERENCES

1. React Documentation. "Describing the UI & Managing State." Meta Platforms, Inc. Available: <https://react.dev>
2. Express.js Guide. "Routing, Middleware, and RESTful APIs." OpenJS Foundation. Available: <https://expressjs.com>
3. MongoDB Documentation. "The MongoDB Manual — Document Databases & Aggregation Pipelines." MongoDB, Inc. Available: <https://www.mongodb.com/docs/>
4. Mongoose Documentation. "Schemas, Models, Validation, and Population." Automattic. Available: <https://mongoosejs.com/docs/>
5. Chart.js Documentation. "Data Visualizations, Canvas Rendering, and Chart Types." Chart.js Open Source Project. Available: <https://www.chartjs.org/docs/>
6. Fielding, Roy Thomas. "Architectural Styles and the Design of Network-based Software Architectures." Ph.D. Dissertation, University of California, Irvine, 2000.
7. Mozilla Developer Network (MDN). "Fetch API, HTTP Status Codes, and Promises." Available: <https://developer.mozilla.org>
8. Vite Documentation. "Next Generation Frontend Tooling." Available: <https://vitejs.dev>
9. Pressman, Roger S., and Bruce R. Maxim. "Software Engineering: A Practitioner's Approach." 9th Edition, McGraw-Hill Education, 2020.
10. W3C Recommendation. "HTML5 & Cascading Style Sheets (CSS3) Specification." World Wide Web Consortium.